

Exercise 1

Task 1.2: list_students.jsp

 **Student Management System**

Student added successfully

[+ Add New Student](#)

ID	Student Code	Full Name	Email	Major	Created At	Actions
7	SV006	Huynh Trinh Phuc Thinh	itcsiu22230@student.hcmiu.edu.vn	CS	2025-11-05 15:47:25.0	Edit Delete
6	SV007	Truong Huy Hoang	itcsiu22228@student.hcmiu.edu.vn	CS	2025-11-05 15:44:22.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-05 14:56:20.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-05 14:56:20.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-05 14:56:20.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-05 14:56:20.0	Edit Delete

1. Database connection

```
try {
    Class.forName("com.mysql.cj.jdbc.Driver");

    conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/student_management",
        "root",
        "Huyhoang12a1"
    );
}
```

I connect to the database student_management by mysql with username and password.

2. Fetch student data

```
stmt = conn.createStatement();
String sql = "SELECT * FROM students ORDER BY id DESC";
rs = stmt.executeQuery(sql);
```

I execute the "String sql = "SELECT * FROM students ORDER BY id DESC";" to get all students from the students table.

3. Display students in a table

```
String studentCode = rs.getString("student_code");
String fullName = rs.getString("full_name");
String email = rs.getString("email");
String major = rs.getString("major");
Timestamp createdAt = rs.getTimestamp("created_at");
```

```
<tr>
    <td><%= id %></td>
    <td><%= studentCode %></td>
    <td><%= fullName %></td>
    <td><%= email != null ? email : "N/A" %></td>
    <td><%= major != null ? major : "N/A" %></td>
    <td><%= createdAt %></td>
</tr>
```

Display results

4. Provide actions

```
<a href="edit_student.jsp?id=<%= id %>" class="action-link"> Edit</a>
<a href="delete_student.jsp?id=<%= id %>"
    class="action-link delete-link"
    onclick="return confirm('Are you sure?')"> Delete</a>
```

edit_student.jsp → allows editing a student.

delete_student.jsp → deletes a student (confirmation prompt).

actions: add, edit, and delete

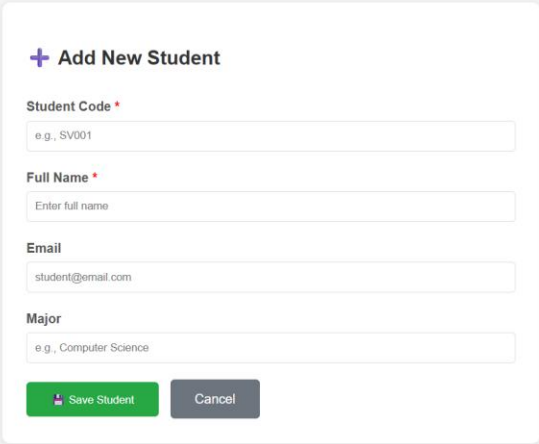
5. Error handling and resource cleanup.

```
} catch (ClassNotFoundException e) {
    out.println("<tr><td colspan='7'>Error: JDBC Driver not found!</td></tr>");
    e.printStackTrace();
} catch (SQLException e) {
    out.println("<tr><td colspan='7'>Database Error: " + e.getMessage() + "</td></tr>");
    e.printStackTrace();
} finally {
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
        if (conn != null) conn.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Display success/error messages and prevent memory leaks.

Exercise 2:

Task 2.1: add_student.jsp



The image shows a web form titled "+ Add New Student" with four input fields: "Student Code" (placeholder: e.g., SV001), "Full Name" (placeholder: Enter full name), "Email" (placeholder: student@email.com), and "Major" (placeholder: e.g., Computer Science). At the bottom are two buttons: "Save Student" (green) and "Cancel" (grey). A small circular icon with a magnifying glass is visible on the right side of the form.

+ Add New Student

Student Code *
e.g., SV001

Full Name *
Enter full name

Email
student@email.com

Major
e.g., Computer Science

 Save Student  Cancel

1. Display add student form

```

<form action="process_add.jsp" method="POST">
  <div class="form-group">
    <label for="student_code">Student Code <span class="required">*</span></label>
    <input type="text" id="student_code" name="student_code"
      placeholder="e.g., SV001" required
      pattern="[A-Z]{2}[0-9]{3,}"
      title="Format: 2 uppercase letters + 3+ digits">
  </div>

  <div class="form-group">
    <label for="full_name">Full Name <span class="required">*</span></label>
    <input type="text" id="full_name" name="full_name"
      placeholder="Enter full name" required>
  </div>

  <div class="form-group">
    <label for="email">Email</label>
    <input type="email" id="email" name="email"
      placeholder="student@email.com">
  </div>

  <div class="form-group">
    <label for="major">Major</label>
    <input type="text" id="major" name="major"
      placeholder="e.g., Computer Science">
  </div>

  <button type="submit" class="btn-submit">Save Student</button>
  <a href="list_students.jsp" class="btn-cancel">Cancel</a>
</form>

```

When I fill this form and clicks “ Save Student”, the data is sent to process_add.jsp using the POST method, and the JSP file will handle inserting the new student into the database.

The form was input with validation, ensuring fields follow the pattern and requiring fields are not empty before submitting.

Form fields:

- Student Code (text, required)
- Full Name (text, required)
- Email (email type, optional)
- Major (text, optional)

2. Action buttons.

```

<button type="submit" class="btn-submit"> Save Student</button>
<a href="list_students.jsp" class="btn-cancel">Cancel</a>

```

Save Student → Submits form to process_add.jsp

Cancel → Returns to the main list page (list_students.jsp) without saving.

2. Visual result.

If successful, the user is redirected to the list page. A green “Student added successfully” message is shown.

Student Management System						
Student added successfully						
+ Add New Student						
ID	Student Code	Full Name	Email	Major	Created At	Actions
9	SV008	Mike	mike@test.com	IT	2025-11-05 22:08:53.0	Edit Delete
7	SV006	Huynh Trinh Phuc Thinh	itcsiu22230@student.hcmiu.edu.vn	CS	2025-11-05 15:47:25.0	Edit Delete
6	SV007	Truong Huy Hoang	itcsiu22228@student.hcmiu.edu.vn	CS	2025-11-05 15:44:22.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-05 14:56:20.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-05 14:56:20.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-05 14:56:20.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-05 14:56:20.0	Edit Delete

If failed, the user stays on this form. A red error box shows what went wrong (from the error parameter).

+ Add New Student

Student code already exists

Student Code *

e.g., SV001

Full Name *

Enter full name

Email

student@email.com

Major

e.g., Computer Science

 Save Student

Cancel

```
<% if (request.getParameter("error") != null) { %>
    <div class="error">
        <%= request.getParameter("error") %>
    </div>
<% } %>
```

Task 2.2: process_add.jsp

1. Get form data

```
String studentCode = request.getParameter("student_code");
String fullName = request.getParameter("full_name");
String email = request.getParameter("email");
String major = request.getParameter("major");
```

Extracts input values sent from the form (add_student.jsp) using request.getParameter().

2. Server-Side Validation

```
if (studentCode == null || studentCode.trim().isEmpty() ||
    fullName == null || fullName.trim().isEmpty()) {
    response.sendRedirect("add_student.jsp?error=Required fields are missing");
    return;
}
```

Ensuring student code and full name are provided.

If validation fails:

Redirects back to the form (add_student.jsp).

Shows an error: "Required fields are missing".

3. Connect to Database

```
try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/student_management",
        "root",
        "Huyhoang12a1"
    );
}
```

Loading MySQL driver and establishing a connection to the student_management database using username root and password Huyhoang12a1.

4. Insert Data Using PreparedStatement

```
String sql = "INSERT INTO students (student_code, full_name, email, major) VALUES (?, ?, ?, ?)";
pstmt = conn.prepareStatement(sql);
pstmt.setString(1, studentCode);
pstmt.setString(2, fullName);
pstmt.setString(3, email);
pstmt.setString(4, major);

int rowsAffected = pstmt.executeUpdate();
```

Uses a PreparedStatement to avoid SQL injection.

The ? placeholders will be filled with user input values.

5. Handle Duplicate Student Code Error

```
} catch (SQLException e) {  
    String errorMsg = e.getMessage();  
    if (errorMsg.contains("Duplicate entry")) {  
        response.sendRedirect("add_student.jsp?error=Student code already exists");  
    } else {  
        response.sendRedirect("add_student.jsp?error=Database error");  
    }  
}
```

If the SQL exception message includes “Duplicate entry”, it means the student_code already exists (violating a unique constraint).

Redirects back to the form with an appropriate error message.

6. Redirect to List Page on Success (To list_students.jsp with success message)

```
if (rowsAffected > 0) {  
    response.sendRedirect("list_students.jsp?message=Student added successfully");  
}
```

If the insertion worked (rows affected > 0):

Redirects to the student list page.

Shows success message “Student added successfully”.

7. Display Error Message on Failure (Back to add_student.jsp with error message)

```
} else {  
    response.sendRedirect("add_student.jsp?error=Failed to add student");  
}
```

If the insert did not succeed (no rows affected):

Redirects back to the form.

Displays an error “Failed to add student”.

8. Close Connections

```
} finally {  
    try {  
        if (pstmt != null) pstmt.close();  
        if (conn != null) conn.close();  
    }  
}
```

Ensures all database resources are properly closed to prevent memory leaks.

Exercise 3:

Task 3.1

 **Edit Student Information**

Student Code

Cannot be changed

Full Name *

Email

Major

 Update

1. Get Student ID from URL Parameter

```
String idParam = request.getParameter("id");
```

Retrieves the id parameter from the request URL

2. Validate the ID Parameter

```
if (idParam == null || idParam.trim().isEmpty()) {  
    response.sendRedirect("list_students.jsp?error=Invalid student ID");  
    return;  
}
```

Checks if the id exists.

Redirects to the student list page with an error if missing.

3. Parse ID to Integer

```
int studentId = 0;
try {
    studentId = Integer.parseInt(idParam);
} catch (NumberFormatException e) {
    response.sendRedirect("list_students.jsp?error=Invalid ID format");
    return;
}
```

Converts the ID string to an integer.

If conversion fails, redirects with an error.

4. Initialize Variables

```
String studentCode = "";
String fullName = "";
String email = "";
String major = "";
```

Creates variables to store data fetched from the database.

5. Connect to Database

```
try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/student_management",
        "root",
        "Huyhoang12a1"
    );
}
```

Loading MySQL driver and establishing a connection to the student_management database using username root and password Huyhoang12a1.

6. Prepare SQL Query

```
String sql = "SELECT * FROM students WHERE id = ?";  
pstmt = conn.prepareStatement(sql);  
pstmt.setInt(1, studentId);  
rs = pstmt.executeQuery();
```

Creates a PreparedStatement to securely query the student by ID.

Prevents SQL injection by using parameter binding.

7. Check if Student Exists and Retrieve Student Data

```
if (rs.next()) {  
    studentCode = rs.getString("student_code");  
    fullName = rs.getString("full_name");  
    email = rs.getString("email");  
    major = rs.getString("major");  
  
    if (email == null) email = "";  
    if (major == null) major = "";  
} else {  
    response.sendRedirect("list_students.jsp?error=Student not found");  
    return;  
}
```

If a student with the given ID exists → proceed.

Otherwise → redirect with “Student not found”.

Extracts data from the result set.

Assigns values to variables for display.

8. Close Database Resources

```
if (rs != null) rs.close();  
if (pstmt != null) pstmt.close();  
if (conn != null) conn.close();
```

Safely closes all database connections and objects.

9. Display HTML Form, Pre-Filled Values, and Form Buttons

```

<form action="process_edit.jsp" method="POST">
  <input type="hidden" name="id" value="<%= studentId %>">

  <div class="form-group">
    <label>Student Code</label>
    <input type="text" name="student_code" value="<%= studentCode %>" readonly>
    <small style="color: #666;">Cannot be changed</small>
  </div>

  <div class="form-group">
    <label>Full Name *</label>
    <input type="text" name="full_name" value="<%= fullName %>" required>
  </div>

  <div class="form-group">
    <label>Email</label>
    <input type="email" name="email" value="<%= email %>">
  </div>

  <div class="form-group">
    <label>Major</label>
    <input type="text" name="major" value="<%= major %>">
  </div>

  <button type="submit" class="btn-submit">Update</button>
  <a href="list_students.jsp" class="btn-cancel">Cancel</a>
</form>

```

Pre-fill form: Load existing student data

Submit: Send updates to process_edit.jsp

Task 3.2: process_edit.jsp

1. Retrieve Student ID and Form Data

```

String idParam = request.getParameter("id");
String fullName = request.getParameter("full_name");
String email = request.getParameter("email");
String major = request.getParameter("major");

```

Collects data sent from the edit form (edit_student.jsp) via POST.

These correspond to form fields and a hidden id.

2. Validate Input

```

if (idParam == null || fullName == null || fullName.trim().isEmpty()) {
    response.sendRedirect("list_students.jsp?error=Invalid data");
    return;
}

```

Checks that required fields exist:

- The id parameter is provided.
- The full_name field is not empty.

If validation fails → redirects back to the list page with an error message.

3. Convert ID to Integer

```
int studentId = Integer.parseInt(idParam);
```

Converts the student ID from string to integer for SQL operations.

4. Establish Database Connection

```
Class.forName("com.mysql.cj.jdbc.Driver");  
conn = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/student_management",  
    "root",  
    "Huyhoang12a1"
```

Loads MySQL driver and connects to the student_management database.

5. Prepare SQL Update Statement

```
String sql = "UPDATE students SET full_name = ?, email = ?, major = ? WHERE id = ?";  
pstmt = conn.prepareStatement(sql);  
pstmt.setString(1, fullName);  
pstmt.setString(2, email);  
pstmt.setString(3, major);  
pstmt.setInt(4, studentId);
```

Uses PreparedStatement for a safe, parameterized update query.

Avoids SQL injection by replacing ? placeholders with user data.

6. Execute Update

```
int rowsAffected = pstmt.executeUpdate();
```

Runs the UPDATE command in the database.

Returns the number of rows changed.

7. Handle Result

```

if (rowsAffected > 0) {
    response.sendRedirect("list_students.jsp?message=Student updated successfully");
} else {
    response.sendRedirect("edit_student.jsp?id=" + studentId + "&error=Update failed");
}

```

If update succeeded ($\text{rowsAffected} > 0$) → redirect to student list with success message.

If no rows updated → redirect back to edit form with error message.

8. Handle Exceptions

```

} catch (Exception e) {
    response.sendRedirect("edit_student.jsp?id=" + studentId + "&error=Error occurred");
    e.printStackTrace();
}

```

Catches any exceptions during database or driver operations.

Redirects back to edit page with a general “Error occurred” message.

9. Clean Up Resources

```

} finally {
    try {
        if (pstmt != null) pstmt.close();
        if (conn != null) conn.close();
    }
}

```

Ensures database objects are closed to prevent resource leaks.

Exercise 4:

Task 4.1: delete_student.jsp

Student Management System

+ Add New Student

ID	Student Code	Full Name	Email	Major	Created At	Actions
9	SV008	Mike	mike@test.com	IT	2025-11-05 22:08:53.0	Edit Delete
7	SV006	Huynh Trinh Phuc Thinh	itcsiu22230@student.hcmiu.edu.vn	CS	2025-11-05 15:47:25.0	Edit Delete
6	SV007	Truong Huy Hoang	itcsiu22228@student.hcmiu.edu.vn	CS	2025-11-05 15:44:22.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-05 14:56:20.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-05 14:56:20.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-05 14:56:20.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-05 14:56:20.0	Edit Delete

1. Get Student ID from URL

```
String idParam = request.getParameter("id");
```

Retrieves the student ID passed in the URL

2. Validate the ID Parameter

```
if (idParam == null || idParam.trim().isEmpty()) {  
    response.sendRedirect("list_students.jsp?error=Invalid ID");  
    return;  
}
```

Ensures the id parameter exists and isn't empty.

If invalid, redirects to the list page with an error.

3. Convert ID to Integer

```
int studentId = Integer.parseInt(idParam);
```

Converts the id string to an integer for SQL operations.

4. Establish Database Connection

```
Class.forName("com.mysql.cj.jdbc.Driver");  
conn = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/student_management",  
    "root",  
    "Huyhoang12a1"
```

Loads the MySQL JDBC driver and connects to the student_management database.

5. Prepare Delete Statement

```
String sql = "DELETE FROM students WHERE id = ?";  
pstmt = conn.prepareStatement(sql);  
pstmt.setInt(1, studentId);
```

Uses PreparedStatement to safely delete a student by ID.

Prevents SQL injection by using placeholders.

6. Execute Delete Operation

```
int rowsAffected = pstmt.executeUpdate();
```

Executes the SQL DELETE statement.

Returns the number of rows affected.

7. Handle Result

```
if (rowsAffected > 0) {  
    response.sendRedirect("list_students.jsp?message=Student deleted successfully");  
} else {  
    response.sendRedirect("list_students.jsp?error=Student not found");  
}
```

- If rowsAffected > 0 → deletion successful
- If 0 → no matching record found (invalid ID or already deleted)

8. Handle SQL Exceptions and General Exceptions

```
catch (SQLException e) {  
    if (e.getMessage().contains("foreign key constraint")) {  
        response.sendRedirect("list_students.jsp?error=Cannot delete: has related records");  
    } else {  
        response.sendRedirect("list_students.jsp?error=Database error");  
    }  
    e.printStackTrace();  
catch (Exception e) {  
    response.sendRedirect("list_students.jsp?error=Error occurred");  
    e.printStackTrace();  
}
```

Catches SQL errors).

If the student has related records, shows a meaningful message.

Catches other unexpected exceptions

Redirects to list page with a generic error message.

9. Close Resources

```
try {  
    if (pstmt != null) pstmt.close();  
    if (conn != null) conn.close();  
}
```

Ensures connection and statement are closed to free resources.

Task 4.2: Modify list_students.jsp to add delete functionality.


```
<a href="edit_student.jsp?id=<%= id %>" class="action-link"> Edit</a>
<a href="delete_student.jsp?id=<%= id %>"
  class="action-link delete-link"
  onclick="return confirm('Are you sure?') "> Delete</a>
```